

Shahjalal University of Science and Technology

Department of Computer Science and Engineering



Bangla BERT Based Close Domain Extractive Question Answering System

NAIMUL HASAN

Reg. No.: 2017331059

4th year, 2nd Semester

MD. SABBIR IRFAN CHY.

Reg. No.: 2017331089

4th year, 2nd Semester

Department of Computer Science and Engineering

Supervisor

DR. MOHAMMAD SHAHIDUR RAHMAN

Professor

Department of Computer Science and Engineering

8th March, 2023

Bangla BERT Based Close Domain Extractive Question Answering System



A Thesis submitted to the Department of Computer Science and Engineering, Shahjalal University of Science and Technology, in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering.

By

Naimul Hasan

Reg. No.: 2017331059

4th year, 2nd Semester

Md. Sabbir Irfan Chy.

Reg. No.: 2017331089

4th year, 2nd Semester

Department of Computer Science and Engineering

Supervisor

DR. MOHAMMAD SHAHIDUR RAHMAN

Professor

Department of Computer Science and Engineering

8th March, 2023

Recommendation Letter from Thesis Supervisor

The thesis entitled *Bangla BERT Based Close Domain Extractive Question Answering System* submitted by the students

1. Naimul Hasan
2. Md. Sabbir Irfan Chy.

is under my supervision. I, hereby, agree that the thesis can be submitted for examination.

Signature of the Supervisor:

Name of the Supervisor: Dr. Mohammad Shahidur Rahman

Date: 8th March, 2023

Certificate of Acceptance of the Thesis

The thesis entitled *Bangla BERT Based Close Domain Extractive Question Answering System* submitted by the students

1. Naimul Hasan
2. Md. Sabbir Irfan Chy.

on 8th March, 2023, hereby, accepted as the partial fulfillment of the requirements for the award of their Bachelor Degrees.

Head of the Dept.	Chairman, Exam. Committee	Supervisor
Md Masum.	Md Masum.	Dr. Mohammad Shahidur
Professor and Head	Professor and Head	Rahman
Department of Computer	Department of Computer	Professor
Science and Engineering	Science and Engineering	Department of Computer
		Science and Engineering

Abstract

In recent years, Question Answering (QA) systems have seen widespread adoption across different businesses, enabling for the provision of around-the-clock online services to customers around the globe. During the Covid-19 outbreak, it became clear how important online medical care, online education, commercial system assistance, and administrative labor are. While many question-answering systems for English have been developed, there is growing interest in developing such systems for other languages, including Bengali. However, due to the language's complex syntax and morphological structure, as well as a scarcity of large-scale annotated datasets, developing a question-answering system in Bengali is a challenging task. In this research work, we developed seven pipelines for close domain questions answering using three BERT based language models and translated version of the SQuAD 2.0 dataset. Our system is made up of two parts: a reader and a retriever. The retriever chooses the best matched context based on the user's question, and reader extracts the response from the context chosen by the retriever. The use of a retriever on top of a reader is due to the fact that BERT based reading comprehension models require two inputs, a context and a question, to assist the user in discovering the desired response. Using BanglaBERT fine-tuned with the SQuAD 2.0 dataset as a reader, a retriever built using BM25 search algorithm to retrieve the top 5 contexts based on a user's question as input, we were able to achieve the best performance out of seven different approaches. We collected 150 passages from the Shahjalal University Of Science and Technology (SUST) website, Wikipedia, various news portals, and websites to build a dataset replicating the shape of SQuAD 2.0 dataset. Using this dataset, we evaluated our proposed model. Our model achieved a notable performance on the task, achieving an Exact Match (EM) score of 49.1% and an F1 score of 62.3%.

Keywords: Question-Answering, Information retrieval, Passage comprehension, BNL

Acknowledgements

We are grateful to the Department of Computer Science and Engineering, Shahjalal University of Science and Technology, Sylhet 3114, Bangladesh, for their assistance with this thesis. We are really grateful to Dr. Shahidur Rahman, Professor of CSE at SUST, for his guidance and being such a generous mentor throughout the semester. The acknowledgment would be incomplete without mentioning all the past researchers in this domain; we appreciate the excellent work of all the prior researchers in this area.

Contents

Abstract	I
Acknowledgements	II
Table of Contents	III
List of Tables	V
List of Figures	VI
1 Introduction	1
1.1 Motivation	3
1.2 Objectives	4
2 Background Study	5
2.1 Question Answering System	5
2.2 Approaches to QA System	7
3 Related Work	9
3.1 Question Answering in Other Languages	9
3.2 Question Answering in Bengali Language	10
4 Datasets	13
4.1 Question Answering in English Language	13
4.2 Question Answering in Bengali Language	15
4.3 Our Dataset : SUSTQuAD	16
4.3.1 Curation of Contexts	17
4.3.2 Question Answer Pair Formation	17

5	Methodology	18
5.1	Retrievers	19
5.1.1	TF-IDF with Cosine Similarity	19
5.1.2	TF-IDF with Bi-gram Hashing	21
5.1.3	Best Match 25 (BM25)	22
5.2	Readers	22
5.2.1	BanglaBERT	23
5.2.2	DistilBERT	24
5.2.3	Multilingual BERT (mBERT)	25
6	Experimental Implementation	26
6.1	Document Store	26
6.2	Fine Tuning	27
6.2.1	Fine Tunning BanglaBERT	27
6.2.2	Fine Tunning DistilBERT	28
6.2.3	Fine Tunning Multilingual BERT	29
6.3	Retrievers	30
6.3.1	TF-IDF with Cosine Similarity	30
6.3.2	BM25Retriever	31
6.4	Pipelines	31
6.4.1	Pipeline 01	32
6.4.2	Pipeline 02	32
6.4.3	Pipeline 03	33
6.4.4	Pipeline 04	33
6.4.5	Pipeline 05	34
6.4.6	Pipeline 06	34
6.4.7	Pipeline 07	35
7	Results	36
7.1	Fine Tuned BERT Models Result	36
7.1.1	Multilingual BERT	36

7.1.2	DistilBERT	37
7.1.3	BanglaBERT	38
7.2	Retrievers Result	38
7.2.1	TF-IDF with Cosine Similarity	38
7.2.2	BM25	39
7.2.3	TF-IDF with Bi-gram Hashing	39
7.3	Pipelines Result	40
8	Conclusion	42
	References	42

List of Tables

4.1	English QA Datasets	14
4.2	Bengali QA Datasets	16
7.1	Training results of Multilingual BERT	36
7.2	Training results of DistilBERT	37
7.3	Training results of BanglaBERT	38
7.4	TF-IDF with Cosine similarity retriever performance	39
7.5	BM25 retriever performance	39
7.6	TF-IDF with Bi-gram Hashing retriever performance	40
7.7	All pipelines result	40

List of Figures

5.1	Proposed Model Structure.	19
5.2	TF-IDF with Cosine Similarity	20
5.3	The DistilBERT model architecture and components.	25
6.1	BanglaBERT Encoder Layer	28
6.2	Pipeline 01	32
6.3	Pipeline 02	32
6.4	Pipeline 03	33
6.5	Pipeline 04	33
6.6	Pipeline 05	34
6.7	Pipeline 06	34
6.8	Pipeline 07	35
7.1	Train/loss of Fine-tuning Multilingual BERT.	37
7.2	Train/loss of Fine-tuning DistilBERT.	37
7.3	Train/loss of Fine-tuning BanglaBERT.	38

Chapter 1

Introduction

Question Answering Systems are computer programs that respond in plain language to a user's questions. They increase the capacity of organizations to connect with individuals and improve user experience or answers. It is both economical and efficient. It has been used in numerous areas, including healthcare, e-commerce, customer service, entertainment, and education. Users often have very precise questions, and they want very particular answers. They would rather not have to learn a special query language, follow a predetermined set of rules for creating queries, or limit their questions to only those within their field of knowledge.

Natural language processing is currently a hot topic because of the advancement of machine learning, deep learning, and transformer-based models, as well as the availability of considerable computational power. There has been a huge increase in the use of online QA services, both text-based and voice-based, during the current epidemic. Closed domain QA systems address issues in a specific domain, such as college admissions, psychology, ticket purchases, and restaurant table reservations etc. . Instead of focusing on a particular domain, open-domain QA systems try to answer users questions regardless of the domain. To answer a user's query, a QA system specializing in a narrow topic must first identify relevant information among a set of predefined documents.

Most modern online QA systems, however, use either a rule-based approach that attempts to find appropriate response by searching a database of pre-existing QA pairings or a list that contains frequently asked questions (FAQ) with their respective solutions. It is very uncommon for users to have a bad experience with the system because it takes too long to obtain the needed results, provides unacceptable results, or redirects them to other websites. Because of the massive amounts of

data often found in service businesses, it is challenging to develop rule-based QA systems capable of answering every conceivable customer question without the assistance of domain specialists. But recent advancement in machine learning and deep learning-based model like Transformer [1] based model has opened a new and efficient way to overcome the problems with rule based, knowledge based or statistical approaches. The development of question-answering systems can benefit greatly from the use of a well structured natural language processing (NLP) model based on the Bengali corpus.

Question answering systems can autonomously reply to human asked questions in natural language. Information retrieval methods can be used to solve textual question-answering tasks, in which we look for documents that answer a given question, extract candidate responses from those documents, and rank them. The goal of a reading comprehension problem in which the solution is found in a passage of text is also applicable to the context-based question-answering task. Reading comprehension models are extremely useful, particularly for industry applications. The reason for this is because a well-trained model of reading comprehension can serve as a question answering system within a limited space. The success of question-answering research can be attributed to the curation of numerous high-quality big reading comprehension datasets and the release of models that perform well on these datasets.

In this research work, we used a system with two parts: a reader and a retriever. Retriever searches all documents and returns only those that are relevant to the question. Reader scans the Retriever's texts and extracts the best answer possibilities. Readers are built on powerful deep learning models, but they analyze text much slower than retriever. We investigate the issue of developing a robust domain-specific QA system. We began by investigating the state-of-the-art pre-trained models for reading comprehension, such as BanglaBERT [2], multilingual BERT [3], and DistilBERT [4], which have proven to be highly effective. QA tasks require context based language processing and efficiently encoding massive textual datasets. Using the translated version of SQuAD 2.0 [5] dataset, all of the suggested models performed well on the question-answering challenge. Our contribution to the study is the development of a robust QA system which can be adapted to any close domain QA task. Also developing a domain specific knowledge base using university website data and a dataset in the form of SQuAD in Bengali.

1.1 Motivation

The number-seven most spoken language in the globe is Bengali [6]. In Bangladesh, nearly 241 million people use it as their primary language [6]. However, despite being the seventh most spoken language, Bengali is still considered a low-resource language due to a lack of documentation and large datasets. Studies on the Bengali language for computational purposes began in Bangladesh in the late 1980s, and they have already produced encouraging findings [7]. But compared to English and other common languages, little is known about the structure of the Bengali language. But Bengali natural language processing has a lot of room for improvement. Researchers in BNLTP can facilitate their job by ensuring that the Bengali language has sufficient linguistic tools and prototypes for BNLTP. A Question Answering system is necessary for all aspects of our digital lives, from e-commerce agents to medical store assistants, frequently asked questions in various organizations, and recipient desks. In this age of advanced computing, the importance of QA systems cannot be overstated. Bengali language there has done some work on question answering. Still, the lack of proper datasets made the researchers suffer, and they did not get the result that which researcher anticipated.

Every year thousands of student from across the country participates sit for the examination to get admission to Shahjalal University of Science & Technology (SUST). So naturally, they have a lot of questions related to the examination and the university itself. The number of questions administrator had to face from the candidates is enormous. For example:

- শাবিগ্রবির সিএসই বিভাগে কয়টা সিট আছে ?
- শাবিগ্রবি কোথায় অবস্থিত ?

Students have to go and ask their queries through helpline or email. Sometimes they don't even get the proper answers. So it will be beneficial if there is a system where the candidates can ask their queries and get the desired information. That is why we are motivated to building a system like this so that our thousands of students can be benefited. But our work is not confined to just one domain. Our proposed model is modular and can be adapted to other domain by training the retriever with context data, related to the desired domain.

1.2 Objectives

- We want to develop a Bengali QA system in close domain.
- Build a system that can be adapted to any other close domain.
- Develop a pipeline consist of reader and retriever.
- Fine tune three BERT [3] based models with translated version of SQuAD 2.0 [5] dataset.
- Try different approaches for developing retriever.
- Compare all the pipelines and select the best approach of retriever reader pipeline.

Chapter 2

Background Study

A question answering system, often a computer program, may create its replies by querying a structured knowledge or information database, typically a knowledge base. Frequently, question answering systems may extract replies from an unstructured corpus of natural language texts. Inadequate openly accessible datasets and resources are a major hindrance to the advancement of close domain bengali QA systems. LUNAR [8] and BASEBALL [9] are the first two well-known QA systems. Early QA methods were rigid, with questions and answers predetermined in advance. Instead of receiving a list of URLs in answer to an inquiry or query typed in natural language (NL), it is necessary to receive a response written in NL. Watson, IBM's first enterprise-level quality QA system, has gained widespread recognition.

2.1 Question Answering System

Question Answering system has been created using various methods. In information retrieval systems, rule-based, Factoid, IR/IE based QA systems etc. are vastly used before the deep learning advancement. Based on this we can categorise the question answering system as follows:

- Factoid : Question-answering is the extraction of concise, appropriate textual responses to natural language queries. As a subclass of QA, factoid QA focuses on queries with semantic responses, such as organization and person related straight forward questions. Important real-world uses for QA systems include search engine optimization and autonomous customer support. For example, In English language some of the factoid questions are,

- What day is it ?
- What is the capital city of Bangladesh ?

In Bengali some of the factorial question are,

- শাবিপ্রবি কম্পিউটার সেন্টার কখন প্রতিষ্ঠিত হয় ?
- শাবিপ্রবিতে কয়টি ডিপার্টমেন্ট আছে ?

- Information Retrieval : The majority of IR QA tools provide a ranked list of documents or passages as answers to a question. The IR based systems are dependent on knowledge base, which demands an addition to the correlated element and general element components in order to process yes-no question types that are included in the text.
- Web Based : The majority of Web based QA solutions make more efforts to generate accurate responses. Though these systems are generally able to handling "WH" type queries, they are unable to generate precise responses. Instead, they retrieve relevant passages containing keywords to retrieve answers out of knowledge base, which requires an additional step to achieve an exact response.
- Rule Based : Rule based QA extends IR based QA. Rule based QA does not require language expertise or complex methods. These systems begin by establishing parse notations, after which they produce training sets and test set by means of a semantic models [10]. This system is made up of different modules, the most notable of which are the Information Retrieval module and the Answer identification or Ranking Module.
- Domain Based :
 - Closed Domain : This kind of system takes the task of a specific domain such as admission, health care, customer care etc. and tries to answer the question related to that field. This system cannot give a proper answer if it is being asked to a domain other than the domain it was fine-tuned for.
 - Open Domain : This is difficult to attain as the system should answer all the questions related to any field.

2.2 Approaches to QA System

The access of a vast collection of information has contributed to the exponential growth in internet usage that has accompanied the development of new technologies. Language processing represents a field of expertise within the larger subject of information extraction, and question answering is a part of this field. Natural language processing (NLP) researchers have been interested in question-answering (QA) systems for decades. From the early days of rule-based systems to the most recent approaches based on deep learning, the evolution of QA systems has been continuous.

In the early days of QA systems, the primary concentration was on rule-based systems, which were designed to recognize patterns and rules in the text in order to answer a question. These systems were limited in their ability to process complex queries and required a significant amount of manual labor to design the rules.

The introduction of machine learning brought about a change in QA system methodology. Utilizing Information Retrieval (IR) techniques, such as TF-IDF and BM25, to retrieve relevant documents and passages from a large corpus of text was one of the earliest methods. The retrieved documents were then utilized to produce a response. While these systems were effective at retrieving pertinent information, they lacked the capacity to answer complex questions and required a significant amount of manual labor to design the IR system.

The introduction of neural networks and deep learning revolutionized the field of natural language processing (NLP), which had an effect on question-answering systems. With the rise of deep learning-based approaches, researchers began developing end-to-end QA systems that could take a query and return an answer without intermediate steps.

Recent emphasis has been placed on developing more sophisticated models that can manage complex questions and generate more accurate answers. These models have been adapted for use in QA systems. The introduction of pre-trained language models, such as BERT [3] and GPT, has revolutionized the field of NLP. Large datasets, such as SQuAD [11], were used to fine-tune these models, which resulted in significant improvements in accuracy.

Another recent trend in QA systems is the use of multi-modal approaches, in which the system is able to process text, image, and video queries. These systems utilize techniques such as visual question answering (VQA) and multi-modal transformers to produce answers that incorporate information from various modalities.

The evolution of QA systems has progressed significantly from rule-based systems to end-to-end deep learning-based models. Recent advancements in pre-trained language models and multi-modal approaches have paved the way for the creation of more precise and robust QA systems.

Chapter 3

Related Work

This section compares and contrasts the effectiveness of different language-specific approaches. In addition, we explore early attempts to develop QA systems in the Bengali language, as well as their limits and obstacles.

3.1 Question Answering in Other Languages

Question-answering systems in various languages have been the subject of substantial research over the years, and new developments in this area of study are continuously being made. The question answer system—categorization, characteristics, categories, and answer extraction has also been studied. Question-answering devices have been created since the 1960s. Some question-answering systems are domain-specific, while others are more general.

According to Day et al. [12], a QA method was developed for the university admission exam in both Japanese and English for history-related subjects. The Japanese question has been translated into English.

Using a knowledge base and information retrieval techniques, Liu et al. [13] created an English QA system. They have evaluated the 9000 questions, nine domains, and nine topics that are included in the 12th standard for the Chinese board. They obtained 70% precision.

Trill [14] is a user interface for a QA system that uses the structured data repository DBPedia. English language support for the question-answering in the Physics Domain (QAPD) technology was developed by Abdi et al. [15]. They utilized NLP, taxonomy, and information retrieval to

attain this result.

Siciliani et al. [16] built a structured repository or database for an Italian question-answering system. The natural language question is translated into data query language. This conversion use NLP. Their system is known as MULIB.

Ketsmur et al. [17] came up with a factoid QA method in Portuguese. On the backend, the structured repository DBPedia is employed. The system's accuracy on 22 Portuguese questions was 67%. After testing the system in English, German, and French, an overall accuracy of 55% was obtained.

Gunawan et al. [18] created a QA method for solving Indonesian-language expressed arithmetic questions. Multiple natural language processing strategies are applied to the English translation of Indonesian to produce mathematical statements. The accuracy of this domain-specific QA method is 80%.

Dhanjal et al. [19] built the Punjabi QA system. Based on the presence of the WH term, questions are divided into nine distinct categories. Then, for each question, potential answers are selected. The database stores long sections of the tenth-grade textbook that have been divided into nine distinct groups, each of which corresponds to a specific style of query. Then QA pairs consisting of question and the potential responses are produced. Mathematical scores are determined by the entity match, total number of matched words, and the tri-gram score. Each question's answers are ranked based on their score. Overall, 91% precision is obtained.

Walia et al. [20] employed RNN and LSTM. As the majority of QA systems are only available in English, the most of the study has focused on the English QA system.

3.2 Question Answering in Bengali Language

Extraction of lexical, syntactic, and semantic characteristics was used by Banerjee et al. [21] to categorize queries into nine different categories. They ran into issues with the large number of interrogative phrases in Bengali comparing to English. There aren't enough trustworthy natural language processing (NLP) tools, and interrogatives can pop up anywhere in a Bengali phrase.

Islam et al. [22] suggested a machine learning strategy utilizing stochastic gradient descent to categorize bengali question-answer types. This research identifies answer types based on words or phrases. The authors great findings in both wide and narrow classes can be attributed to the small

size of the corpus used to evaluate the recommended models and the lack of descriptive questions in the proposed corpus, which together led to insufficient variation in the data.

Niroy et al. [23] also constructed a question classification system utilizing feature extraction methods that were quite similar to those described in Banerjee et al. [21], in addition to employing support vector machines that used uni-grams as a feature.

Khan et al. [24] suggested in 2018 a Bengali QA system that determines anaphora and cataphora for straightforward utterances. To validate the suggested system strategy, semantic and grammatical evaluations were carried out on this model. The question was turned into a series of phrases, and any "wh" words were taken out. The recommended question-answering method had a 74% success rate when it came to answering the exam questions correctly. The writers of this study concentrated on improving answer extraction approaches by employing anaphora-cataphora resolution techniques. The document corpus used to assess the proposed models was too small. It lacked complex and descriptive terms to provide significant document data variations, even though their suggested method yielded reasonable accuracy.

Using lexical and semantic analysis, Banerjee et al. [25] established a model for answering questions in Bengali based on their findings. The lexical analyzer comes first. This component disassembles the input phrases into their component tokens, which are then subjected to content-sensitive grammar (CSG) analysis. The data dictionary is the second phase, comprising many different kinds of POS tagging data. It assigns a tag to the POS system for each Bengali token. The development of rules is the third phase, and during this phase, the grammar for parsing tokens was constructed. A semantic analyzer is the fifth stage, and it is responsible for estimating the semantic similarities that are accessible on the parse tree that was constructed. The last phase is called the evaluator, which judges how well the retrieved response matches the input text.

For closed domain of Shahjalal University of Science and Technology (SUST), Sarker et al. [26] suggested a factoid question-answering system to help applicants with admission exams. For categorizing questions and documents, the writers settled on a set of five broad classes. The author deleted stop words and special characters, tokenized the words in unusual cases and fixed writing mistakes by hand as part of the preparation processes. They employed FastText [27] as an embedding approach, a CNN classifier for document classification, and a vector space model and edit distance [28] methods for extracting answers. A SVM with a linear kernel produced the high-

est accuracy for question categorization at 90.6%. The precision for document categorization was 75.3%. Without knowing the object's name, the response extraction method was only about 56.8% accurate, but with the name it was about 66.2%. A document hit rate of around 72% was obtained with the proposed system. This article developed a generic bengali factoid question-answering system. The authors classified questions and extracted replies using a variety of methodologies despite this, the suggested question responding system can only provide answers related to SUST.

An informative bengali question-answering system was proposed in 2019 by Kowsher et al. [29]. The authors have limited their proposed system to the specific context of Noakhali Science and Technology University. The authors employed various preprocessing strategies, including anaphora, punctuation and exceptional character removal, stop word removal, verb processing, lemmatization, and synonym processing. Users' queries were answered using a combination of the Jaccard similarity, Cosine similarity, and Naive Bayes method. In order to cut down on storage requirements and processing times, a support vector machine was also employed. In this case, we found that cosine similarity (93.22%) outperformed jaccard similarity (82.44%) and the naïve bayes classifier (91.46%). In order to research this study, a Bengali intelligence bot was deployed. Although the author's suggested system achieved satisfactory accuracy, the proposed model was trained on data from just the NSTU domain. The quality of this study would increase if its authors chose and tested more domains using the suggested framework.

Islam et al. [30] suggested the system that distinguishes the question type and gives appropriate responses for measurement of time and quantity-related questions; otherwise, it finds relevant responses depending on the questions. The author has eliminated stop words from the questions and conducted stemming on questions and selected documents. Here, the average accuracy was 0.35, the average recall was 0.65, and the F1 score was 0.45. Multiple documents were utilized to develop a question-answering system in this article.

Chapter 4

Datasets

Deep Learning models are data-hungry and require huge amount of data for developing the most accurate system. Even if you’ve built excellent algorithms for machine learning models, data quality is just as important as data quantity.

4.1 Question Answering in English Language

Deep Read [31] and the MCTest are examples of early reading comprehension tasks. We found out that, numerous try for curating large-scale QA corpus for English, including CNN/ Daily Mail Corpus , SQuAD 1.0 [11], SQuAD 2.0 [5], CoQA [32]. Domain specific QA datasets such as NewsQA [33] and TriviaQA [34] are also accessible for English language. SQuAD 1.1 [11] and SQuAD 2.0 [5] are commonly used as a baseline, and incorporated into NLP benchmark leader boards like as GLUE [35] and SUPERGLUE [36]. Due to the scope and depth of the question-answering literature, several survey and review papers on this issue have been authored recently. There are other variations of question-answer tasks, such as reading comprehension, in which the objective is to extract a response from a paragraph. Open domain question-answering, in which the input is a large amount of unstructured text, and multi-modal question answering, in which the context might include additional media such as video, image, and audio, are among the several forms of question answering. In place of text alone, Common sense QA assignments in which answers require common sense or world knowledge, and complex reasoning QA task in which reasoning skills are required.

Both the questions and the contextual readings are presented in a variety of languages. Furthermore, there might be other subdivisions within the three main groups of corpus, questions, and responses. The pieces that make up a corpus may be anything from a single passage to a collection of them, a document, a URL, a paragraph illustrated with images, a series of photographs, or even a short video clip. Questions can be either natural text or synthetic (a list of words, like "the president of the Bangladesh resides in " rather than a grammatical question, like "Who is the president of Bangladesh ? "). The response may be a fragment taken from the passage in context (with the stipulation that the fragment itself be a part of the passage in context), it can be free-form (the answer need not be a part of the passage), or it can be a multiple-choice answer with yes/no options. The datasets also vary in quantity, source, complexity, and metrics for evaluation.

Table 4.1: English QA Datasets

Task	Dataset	Data Source	Number Of Sample	Metric
Reading comprehension	SQuAD 1.1 [11]	Wikipedia	100k	EM, F1
	SQuAD 2.0 [5]	Wikipedia	100k	EM, F1
	BookTest	Stories	14,160,825	Accuracy
Multihop RC	HotpotQA	Wikipedia	113k	EM, F1
	MultiRC	NEWS	6k	EM, F1
Natural Thinking	ARC	School Science Curriculum	5k+	EM, F1
	MCScript	Narrative Text	13,939	Accuracy
	ReCoRD	News	120, 730	EM, F1
Complex Reasoning	DROP	Wikipedia	96,567	EM, F1
	Google MC-AFP	Gigaword	1,742,618	Accuracy
	TriviaQA [34]	Bing	10,022	Accuracy
Domain Specific	PaperQA	Scientific Paper	80k+	Accuracy, F1
	ReviewQA	Resturent Com-ments	587492	Accuracy

Task	Dataset	Data Source	Number Of Sample	Metric
Conversational RC	DREAM	English Exam	10k+	Accuracy
	CoQA [32]	jeopardy	127k	EM, F1
	QuAC	Healthcare	98,407	F1, HEQ
Open Domain QA	MCTest	Factoid	2000+	Accuracy
	Quasar	Stack-overflow	37k+, 43 k+	Accuracy, F1
	SearchQA	archive and Google	140,461	Accuracy, F1
	WikiQA	Bing, Wikipedia	3047	Precision

4.2 Question Answering in Bengali Language

Bengali question-answering research is in its infancy compared to English question answering research owing to a lack of essential language processing technologies and a dearth of data. Most Bengali QA research has concentrated on constructing question categorization systems as part of a larger QA system or building factoid-like QA systems with an emphasis on information retrieval.

Table 4.2: Bengali QA Datasets

Papers	Information Provided by the Document	Task
Khan et al. [24]	Out of 50 documents, 50 positive sentences were chosen.	Improving the Bangla QA system’s answer extraction method
Islam et al.[22]	Documents chosen at random. An additional 3,334 files were uploaded.	The Bangla QA system now uses a word or phrase-based response type classification.
Uddin et al. [37]	History-related questions were used to produce 11,500 question-answer pairings.	Paraphrased QA system of Bangla.
Sarker et al. [26]	220 documents and 15,355 questions.	Bangla QA system for Factoid Question.
kowsher et al. [29]	Test and training data were selected at almost 1:1 ratio from 74 topics	Bangla QA system based on intelligence.

4.3 Our Dataset : SUSTQuAD

BERT deep learning models are trained and fine-tuned on Question Answering task using the QA dataset (SQuAD 2.0) [5], a reading comprehension dataset including over 100k+ questions collected from articles. Each response to these questions consists of a paragraph from the associated passage. Each SQuAD 2.0 [5] sample comprises of three primary elements: a referenced text, questions, and answer. The objective is to get the solution to a given question from the cited passage. So, the paragraph and question are what the deep learning model gets as input, and the response phrase in the paragraph is what it predicts the label to be. Our dataset was made by following the structure of SQuAD 2.0 [5]. We gathered our data in two stages: curating contexts and question-answer pair formation about those contexts.

4.3.1 Curation of Contexts

To retrieve contexts, we used Google Translate to translate the Shahjalal University of Science and Technology websites. We extracted individual context from each of translated web pages, removing images, figures, tables, and short contexts. We also gathered information about Shahjalal University of Science and Technology from Wikipedia entries, news portals, and blog websites. The end result was 150 context covering a broad range of Shahjalal University of Science and Technology-related topics from academic advice to admissions requirements.

4.3.2 Question Answer Pair Formation

Every context was read in its entirety, and at most 5 questions were generated from each context. There were no restrictions placed on the questions in terms of their semantics or syntax. Each answer had to provide a condensed version of the relevant context to fill in the gap. There were no requirements placed on the questions' wording to match the context, and any variations were accepted in order to better direct the model's training. In this way, we collected a total of five hundred question-answer combinations. One such instance:

Context : দীর্ঘ সংগ্রাম এবং একটি অবিচ্ছিন্ন আন্দোলনের পর ১৯৮৬ সালে শাহজালাল বিজ্ঞান ও প্রযুক্তি বিশ্ববিদ্যালয় (শাবিপ্রবি) প্রতিষ্ঠিত হয়। সেই সময়ে এই ধরনের একমাত্র বিশ্ববিদ্যালয় এটি, পহেলা ফাল্গুন ১৩৯৭ বঙ্গাব্দ (১৩ ই ফেব্রুয়ারি, ১৯৯১) এর কর্মজীবন শুরু করে তিনটি বিভাগ: পদার্থবিজ্ঞান, রসায়ন ও অর্থনীতি, ১৩ জন শিক্ষক এবং ২০৫ জন শিক্ষার্থী নিয়ে। যা এখন ৭ টি অনুষদ, ২৭ টি বিভাগ এবং দুটি ইনস্টিটিউটে প্রসারিত হয়েছে। শিক্ষকদের সংখ্যা ৫৬৬ এবং শিক্ষার্থীদের ৮৫৯৬ পর্যন্ত বৃদ্ধি পেয়েছে। পাশাপাশি বিশ্ববিদ্যালয়ের শিক্ষার্থীর সাথে ১২ টি মেডিকেল কলেজের ৪০০০ জন শিক্ষার্থী সংযুক্ত রয়েছে [38] ।

Question : শাহজালাল বিজ্ঞান ও প্রযুক্তি বিশ্ববিদ্যালয় (শাবিপ্রবি) কবে প্রতিষ্ঠিত হয়?

Answer: ১৯৮৬ (1986) Question : বর্তমানে শাহজালাল বিজ্ঞান ও প্রযুক্তি বিশ্ববিদ্যালয়ের শিক্ষার্থী সংখ্যা কত?

Answer : ৮৫৯৬ (8596)

Question : শাহজালাল বিজ্ঞান ও প্রযুক্তি বিশ্ববিদ্যালয়ের সাথে সংযুক্ত মেডিকেল কলেজের কতজন শিক্ষার্থী রয়েছে?

Answer: ৪০০০

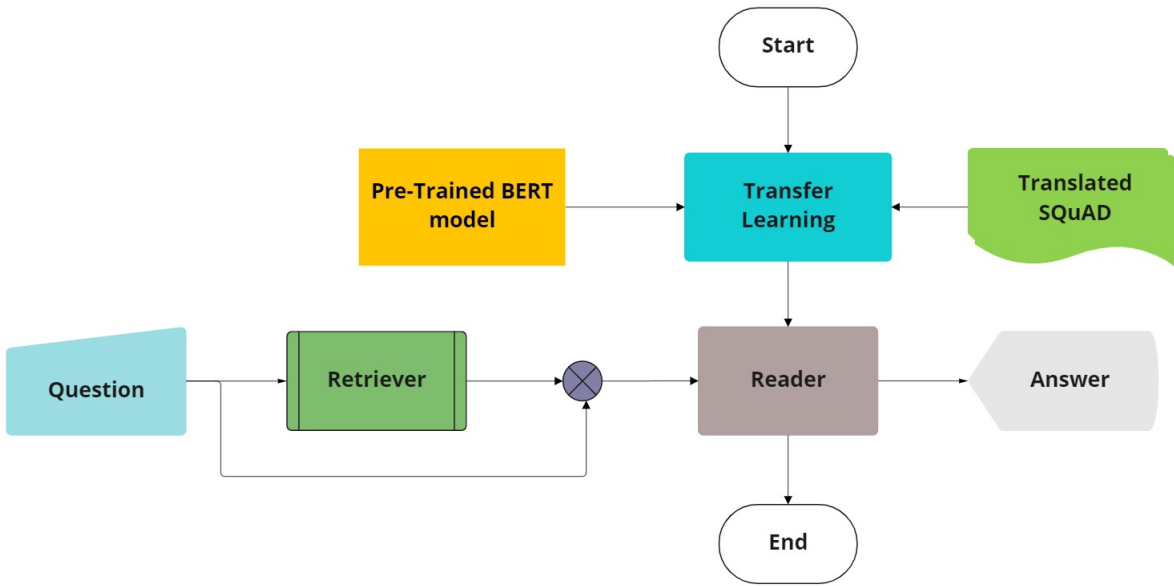
Chapter 5

Methodology

For building close domain question answering (QA) systems we are proposing to use the Retriever-Reader approach. In this method, two distinct models are trained to accomplish different task: retriever and reader.

Based on the input query, the retriever model selects related contexts from a large corpus of text data. This is accomplished by ranking the documents based on their relevance to the question using information retrieval methods such as TF-IDF or BM25. The reader model is then fed the top-ranked contexts. The reader model is in charge of reading and comprehending the chosen contexts, as well as extracting pertinent information to answer the question.

The retriever-reader approach has several benefits over other close domain QA methodologies. It is scalable because it can handle large amounts of text data and rapidly select pertinent contexts. It also provides flexibility because different retriever and reader models can be used based on the domain or task. In this part, the phases of our planned endeavor are broken down in further depth. A flowchart illustrating the procedures of the extractive QA system can be seen in Fig. 5.1 .



miro

Figure 5.1: Proposed Model Structure.

5.1 Retrievers

The retriever model is accountable for selecting pertinent documents from a large corpus of text data based on the query input. In this research work, we employed three distinct retriever design strategies. These methods are outlined below.

5.1.1 TF-IDF with Cosine Similarity

To retrieve a context using TF-IDF with cosine similarity, we used all of the dataset's contexts. When a question is entered, the TF-IDF value of all contexts against the input question is calculated. The result is then passed to the cosine similarity function, which calculates the cosine similarity. The context that best matches is chosen as the context.

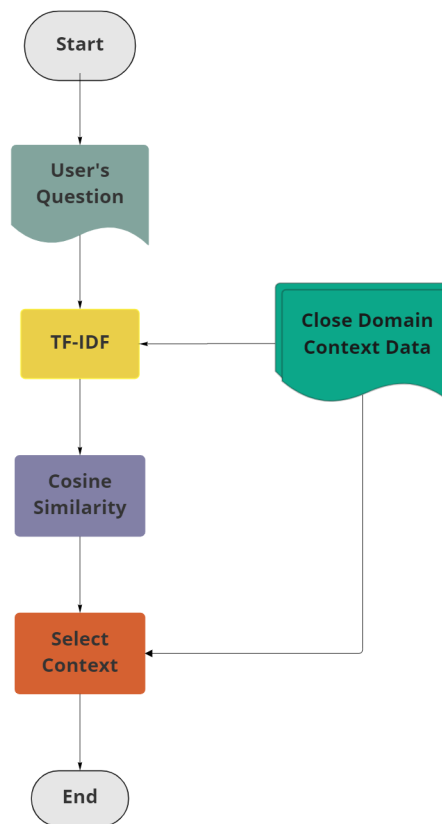


Figure 5.2: TF-IDF with Cosine Similarity

5.1.1.1 TF-IDF

The term frequency-inverse document frequency (TF-IDF) is a weighting method used in information retrieval and natural language processing. It is used to determine the importance of a word in a collection of documents.

The term frequency (TF) component of the formula calculates the frequency of a word in a given text. It calculates how frequently a word appears in a document and assigns a weight to it appropriately. This aids in determining the most essential words in a document.

The inverse document frequency (IDF) component of the formula determines how frequent or infrequent the occurrence of a particular word is across the entirety of the document collection. This aids in identifying and weighting words that appear frequently in multiple documents.

The sum of the TF and IDF values assigns a weight to each word in a document, indicating

its significance. This weight can be used to rank documents based on how relevant they are to a query. Documents with more instances of the query terms and higher TF-IDF scores are thought to be more significant to the query.

Overall, TF-IDF is a useful technique for information retrieval and text analysis because it aids in the identification of important words while filtering out common words that may be irrelevant to the task at hand.

5.1.1.2 Cosine Similarity

The cosine similarity metric is used to compare the similarity of two vectors in a high dimensional environment. It is frequently used to compare the similarity of two documents, words, or phrases in information retrieval and language processing. To compute cosine similarity, we first represent each vector in high-dimensional space as a list of values indicating the vector's magnitude and direction along each dimension. The dot product of the two vectors is then computed, which is the sum of the products of the respective values in each vector. Finally, we split the dot product by the magnitude product of the two vectors. The resulting number is the cosine of the angle formed by the two vectors, indicating their similarity. The cosine similarity value ranges from -1 to 1, with -1 representing total dissimilarity, 0 representing no similarity, and 1 representing complete similarity. A higher cosine similarity score shows that the two vectors are more similar.

5.1.2 TF-IDF with Bi-gram Hashing

TF-IDF is a popular method in information retrieval. It is a statistical measure that considers the frequency of a term in a document (term frequency, or TF) as well as the inverse document frequency (IDF), which evaluates the importance of a term across all documents in a collection. Instead of using single terms as the foundation for TF-IDF, bi-gram hashing divides the text into pairs of consecutive words (bi-grams). Using a hashing algorithm, these bi-grams are then hashed into a fixed-length vector. The following phases are involved in the TF-IDF with bi-gram hashing for document retrieval process :

1. Tokenization : Tokenization is the process of breaking down a document into individual words or bi-grams.

2. Hashing : Using a hashing algorithm, individual words or bi=grams are hashed into a fixed-length vector.
3. Inverse Document Frequency (IDF) : The frequency of each word in the document is calculated and normalized by the total number of terms in the document.
4. Inverse Document Frequency (IDF) : Each term's significance within the collection is calculated by taking the logarithm of the relation between the total number of documents and the number of documents that contain the term.
5. TF-IDF weighting: The ultimate weight of each term in the document is determined by multiplying its TF by its IDF.

By capturing the context of adjacent words in the text, bi-gram hashing with TF-IDF can increase the accuracy of document retrieval. It may, however, result in a greater number of features, increasing the computational intricacy of the retrieval process.

5.1.3 Best Match 25 (BM25)

The BM25 algorithm is a famous information retrieval algorithm that has gained popularity because of its effectiveness in ranking search results. It is well-known for its effectiveness in ranking search results and has been extensively used in many search engines. The BM25 algorithm is a variation on the TF-IDF weighting system used in information retrieval. Robertson and Zaragoza suggested it in 2009 as an improvement over the previous BM11 algorithm. The BM25 algorithm uses a probabilistic model to compute a document's relevance score based on the frequency of query terms in the document and the inverse frequency of those terms in the collection of documents. The algorithm considers the length of the document, the length of the question, and the average length of the collection's documents.

5.2 Readers

In the retriever-reader approach for close domain question answering, the reader model is responsible for reading and comprehending the retrieved contexts and generating the final response to the user's question. The retriever model is responsible for retrieving the contexts. Following

the retrieval of a group of pertinent contexts from the document collection by the retriever component, the reader model takes these contexts as input and, based on its comprehension of both the question and the context, chooses the answer that is most likely to be correct. When it comes to carrying out this task, the reader model will generally rely on machine learning strategies such as deep neural networks and attention mechanisms. The reader model is trained to make connections between relevant situations and their corresponding responses by being exposed to a big dataset of question-answer pairs. During the inference process, the reader model receives as input the retrieved contexts as well as the question posed by the user. It then generates a prediction of the response that is most likely to be correct based on the associations it has learned to make.

5.2.1 BanglaBERT

BanglaBERT [2] is a transformer-based Bengali language model that has been pre-trained. It is based on the Google developed BERT [3] architecture, which is designed to learn deep bidirectional text representations. BanglaBERT follows the same fundamental structure as the original BERT model, with modifications to accommodate the Bengali language's unique characteristics. Multiple layers of self-attention and feed-forward neural networks compose the model. BanglaBERT, like other pre-trained language models, has been trained on a large corpus of Bengali text data in order to gain a basic understanding of the language. This pre-training permits the model to be fine-tuned for a variety of natural language processing tasks, such as question answering, text classification, and named entity recognition. Input to BanglaBERT consists of a sequence of word embeddings, which are learned representations of each vocabulary term. These embeddings are fed to the model's self-attention layers, allowing to capture the contextual relationships between words in the input sequence. The self-attention layers are followed by feed-forward neural networks, which handle the self-attention layers' output further. These layers contain a non-linear activation function, such as the Rectified Linear Unit (ReLU), enabling the model to capture complex non-linear relationships between input features. Like BERT [3] BanglaBERT [2] employs a modified tokenization strategy that takes into consideration the distinct Bengali characters and symbols. Important for tasks such as language generation and text summarization, the model also includes a special token that represents the beginning and end of sentences.

Overall, BanglaBERT's structure is made to capture the intricate connections between words

and phrases in Bengali text, and it has already been used in several studies with promising results in a wide range of NLP tasks.

5.2.2 DistilBERT

DistilBERT [4] is a version of the BERT [3] language model that is smaller and runs faster. The architecture of the BERT language model is the basis for the structure of DistilBERT. However, DistilBERT is a smaller and more efficient version of BERT. Like BERT [3], DistilBERT is made up of many layers of self-attention and feed-forward neural networks. It has a simpler design that cuts down on the number of parameters in the model. In particular, DistilBERT uses a method called "knowledge distillation" to turn the larger BERT model into a smaller, more efficient version. This is done by training a smaller model (DistilBERT) to act like the larger model (BERT). This lets the smaller model capture the same general-purpose language representations as the larger model. Even though DistilBERT is smaller than BERT, it has been shown to perform as well as BERT at a number of NLP tasks, such as classifying text and answering questions. Its efficiency makes it useful for applications on mobile devices or edge devices that don't have a lot of computing power.

Overall, DistilBERT is built to give high-quality representations of language while using as few computational resources as possible. This makes it a useful tool for a wide range of NLP applications. It is a useful addition to the growing ecosystem of pre-trained language models, making NLP tasks that don't need the full power of bigger models like BERT.

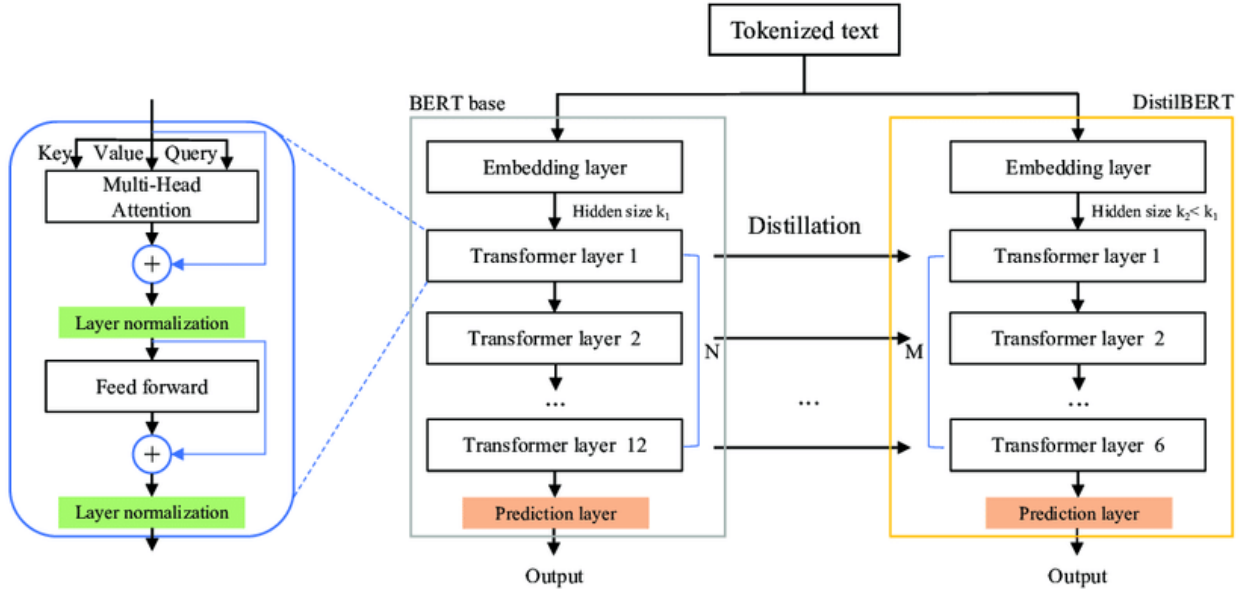


Figure 5.3: The DistilBERT model architecture and components.

5.2.3 Multilingual BERT (mBERT)

Multilingual BERT is abbreviated as mBERT. It is a pre-trained language model that Google developed. This model is able to comprehend and generate text in a variety of languages. The mBERT is trained on a large text corpus from 104 different languages. Google's original BERT [3] language model is the basis for the structure of mBERT (Multilingual BERT). Like BERT, mBERT uses a Transformer [1] architecture to learn general-purpose language representations. The mBERT is different because it was trained on a large collection of text from 104 different languages, both Indo-European and non-Indo-European. This enables the system to recognize cross-lingual relationships between words and phrases and generate text in multiple languages. The first step is a self-attention layer that figures out how important each word in the text is compared to all the other words. Its ability to understand more than one language makes it useful for multilingual applications and tasks that involve code-switching or text in more than one language.

As a whole, mBERT is designed to supply excellent language approximations for many different languages. Because of this, it can be used effectively in multilingual natural language processing tasks.

Chapter 6

Experimental Implementation

We developed 7 pipelines using the retrievers and readers described in the methodology part of this research. To use the readers we have to fine-tune them for question answering task first. Then using the retrievers we developed our pipelines. We used the Haystack (an NLP framework to Use transformers) and Huggingface libraries to train and test our models and pipelines.

6.1 Document Store

We have used Haystack ElasticsearchDocumentStore to store all the contexts. ElasticsearchDocumentStore is a module in the Haystack framework that is used to store and access documents in an Elasticsearch index. ElasticsearchDocumentStore is a document store that utilizes Elasticsearch as its underlying search engine. It is well-suited for use cases such as question-answering systems and chatbots because it offers an efficient and scalable solution for storing and searching big collections of documents. Documents are indexed in Elasticsearch using their unique IDs when using ElasticsearchDocumentStore, making it simple to access them based on their ID or other search criteria. The papers can be saved in a variety of formats, such as JSON, CSV, and plain text. ElasticsearchDocumentStore also includes features for customizing the indexing and retrieval process, such as the ability to designate the index name and settings, determine the document schema, and control search behavior. This enables users to fine-tune the search engine to their specific needs.

Overall, ElasticsearchDocumentStore is a helpful module in the Haystack architecture for scalable and efficient document storage and retrieval using Elasticsearch as the underlying search en-

gine. It offers a flexible and adaptable option for developing question-answering systems and other applications that require efficient textual data search and retrieval.

6.2 Fine Tuning

We started with fine tuning BERT based pretrained model BanglaBERT [2], which can comprehend bengali, and continued to fine tune two more model based on BERT, one is DistilBERT [4] and the other one is mBERT [3]. Comparing to the pre-training, the computational cost of fine-tuning a BERT based model on downstream task is low. We fine-tuned the three models using the translated version of the SQuAD 2.0 [5] dataset. We make use of Google Colab and Kaggle Notebooks. We only trained for one iteration because these are large models and it was obvious that we need more processing capacity in order to train the model for more than 3 or 5 epochs. The training period for only one session in kaggle and colab notebooks was more than four hours long with the assistance of GPUs.

6.2.1 Fine Tuning BanglaBERT

Before giving a response, BanglaBERT [2] must determine which sentence, paragraph, or section of the given text includes the answers. This is depicted by giving each token in the text or document a certain probability of being labeled as the start of the solution and the same probability of being labeled as the end of the solution. If the response is more than one word long and has a distinct beginning and ending token, then the solution is those tokens in between. In order to train a linear layer for question answering, one must first train the BanglaBERT [2] model with appropriately calibrated parameters. An additional linear classification layer was used for fine-tuning the model by feeding the embedding for the [CLS] token of each sequence. The Adam optimizer was employed and the dropout probability was held constant at 0.1. 16 batch size is used on training and evaluation, with $2e^{-5}$ learning rate, we calibrated the BanglaBERT for question-answering tasks. The question and referenced text are combined into a single string, separated only by the [SEP] token.

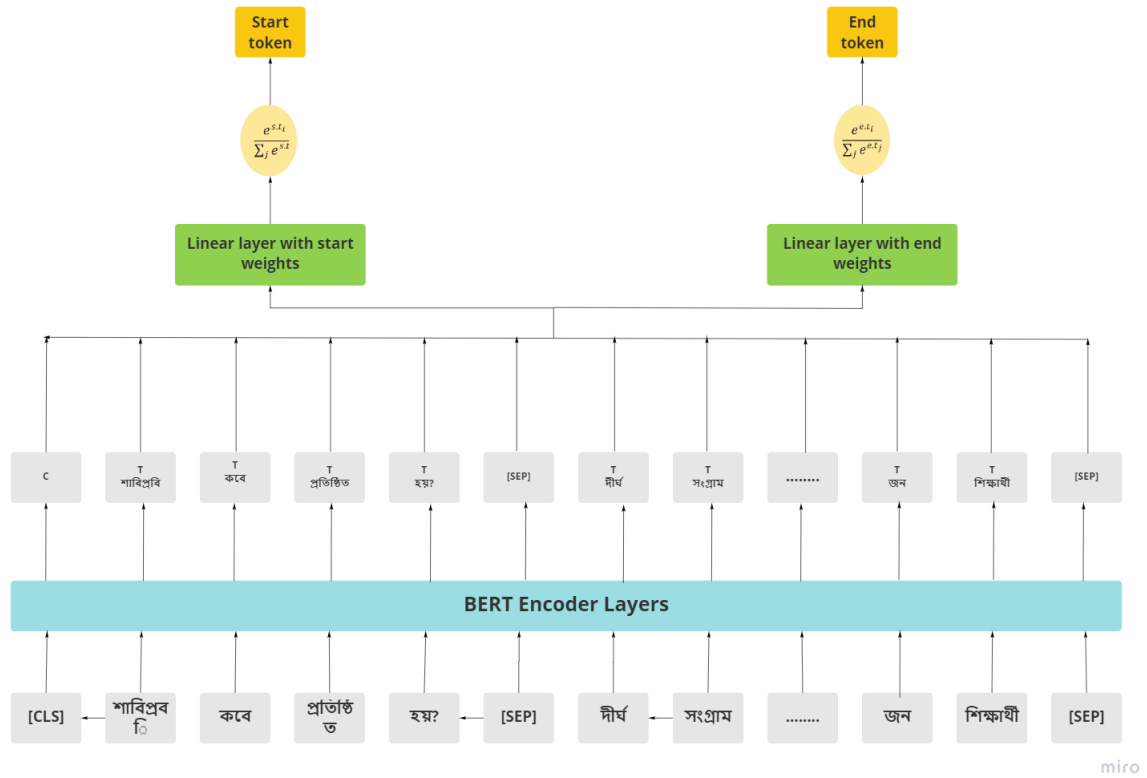


Figure 6.1: BanglaBERT Encoder Layer

The question in BanglaBERT [2] is represented by an A segment embedding, while the reference text is represented by a B segment embedding. The beginning and ending token classifiers receive the embeddings that were generated by the BanglaBERT model's final transformer encoder. A singular collection of weights is given to the embedding of each token for both the initial and final token classifiers. Tokens with the greatest probability of being the start and end tokens of the response in the text are returned. There are currently 118,000 QA combinations in the SQuAD collection.

6.2.2 Fine Tuning DistilBERT

To fine-tune DistilBERT [4] for a specific task, the model must be trained on a task-specific dataset in order to modify its parameters to that task. In the case of question-answering, the goal is to train DistilBERT to output the correct response to a given query based on a given passage of context. The input consists of a query and a context-provided passage. The query and context pas-

sage are tokenized and encoded with the WordPiece tokenizer. The concatenated embeddings are then fed into the model. Each Transformer encoder layer is made up of a self-attention mechanism and a feed-forward neural network. For the question-answering task, the final output layers are used to determine the beginning and ending positions of the answer span within the context passage. During training, the model is taught to pay attention to different parts of the input sequence, allowing it to understand the relationship between the query and the context passage. These output representations are then fed into a set of feed-forward neural networks, which predict the beginning and ending positions of the answer span within the context passage. Two Softmax classifiers are used to predict the likelihood that each token in the sequence corresponds to the beginning and end positions of the answer span. Throughout training, the model is optimized to minimize the cross-entropy loss between the predicted and actual answer span start and end positions.

During training, hyperparameters are employed to specify the settings and conditions for the distilBERT [4] model to learn and optimize on the translated SQuAD 2.0 [5] dataset. In this instance, the learning rate was set to $2e^{-5}$, which dictates the model's parameter update size during training. The train batch size and evaluation batch size parameters were both set to 16 to establish the number of samples processed in each training and evaluation batch. The seed was set to 42, which ensures that the results can be replicated. Adam with betas=(0.9, 0.999) and $\epsilon = 1e^{-8}$ was used for training, which dictates how the model updates its parameters based on the gradients calculated during back propagation. A linear learning rate scheduler was also used, which decreases the learning rate throughout the training period. The number of epochs was specified to 1, which determines how many times the model iterates over the entire training dataset.

6.2.3 Fine Tunning Multilingual BERT

The pre-trained mBERT [3] model is fine-tuned using the translated SQuAD 2.0 [5] dataset, which contains a significant number of question-answer pairs from multiple domains. Throughout the process of fine-tuning, the model is trained to predict the correct response to a given query based on the input text passage. First, the input embeddings layer maps each input token to a high-dimensional vector representation in order to convert the input tokens into an embeddings set. Then, these embeddings are fed into multiple transformer blocks, which execute self-attention operations on the embeddings. The self-attention mechanism enables the model to evaluate the

significance of each token in relation to the others, enabling it to encode complex dependencies and inter-word relationships. The output layer of mBERT for the question-answering task applies two distinct linear transformations to the final layer of the transformer block. One linear transformation produces a probability distribution over the start positions of the answer, while the other produces a probability distribution over the end positions. These probabilities are then used to identify the portion of the input text that constitutes the answer to the given query. Throughout training, the model learns to optimize the parameters in order to minimize the loss function, which measures the difference between the predicted and actual answers. This process entails back propagating the model's errors through its layers and adjusting the weights accordingly. The procedure is repeated until the model's accuracy reaches an acceptable level.

In order to fine-tune the mBERT model for question answering, task-specific hyperparameters were selected for training. Along with a train batch size and evaluation batch size of 16, a learning rate of $2e^{-5}$ was utilized. Additionally, a seed value of 42 was set to assure the reproducibility of the results. Adam optimizer with $\text{betas}=(0.9, 0.999)$ and $\epsilon = 1e^{-8}$ was used as its effectiveness in deep learning tasks has been demonstrated. During training, a linear learning rate scheduler was also used to modify the learning rate. Ultimately, the model was trained for one epoch, which means that it was exposed to the entire training dataset only once. Based on previous research and experimentation, these hyperparameters were chosen to obtain the best possible performance of the mBERT model for question answering on the translated SQuAD 2.0 dataset.

6.3 Retrievers

We have used four different types of retrievers to make our pipelines. In this case we have used the Haystack framework which provides us the builtin components to build the retrievers. We have used the TfidfRetriever and BM25Retriever from Haystack to implement our desired retrievers.

6.3.1 TF-IDF with Cosine Similarity

TfidfRetriever is a module in the Haystack framework that retrieves relevant documents from a collection of documents using the TF-IDF (term frequency-inverse document frequency) weighting scheme in response to a user's inquiry. TfidfRetriever is a retriever implementation that employs

the `TfidfVectorizer` to convert raw text data into a numerical format, and then calculates the cosine similarity between the query vector and each document vector to rank the documents according to their relevance to the query. The most pertinent documents are then returned as a consequence of the search. `TfidfRetriever` also provides a variety of customization options for the retrieval process, such as the ability to specify the maximum number of documents to retrieve, determine the minimum and maximum document frequency for the vocabulary, and select a subset of the corpus for training the vectorizer. `TfidfRetriever` is a valuable module of the Haystack framework for retrieving relevant documents from a collection of documents using the TF-IDF weighting scheme in response to a user's query.

6.3.2 BM25Retriever

The `BM25Retriever` module in the Haystack framework employs the BM25 (Best Matching 25) ranking method to obtain documents from an accumulation of documents in response to a user inquiry. `BM25Retriever` is a retriever implementation that uses the BM25 algorithm to compute the relevance score of each document in relation to the query. The BM25 algorithm is a probabilistic ranking function that estimates the relevance of a document based on the frequency of terms and its length. It considers the frequency of each query term in the document, as well as the length of the document and the aggregate frequency of terms in the corpus. We can customize the retrieval process with `BM25Retriever` by specifying the maximum number of documents to retrieve, controlling the hyperparameters of the BM25 algorithm, and selecting a subset of the corpus to train the vectorizer. Overall, `BM25Retriever` is a useful Haystack framework module for retrieving relevant documents from a collection of documents in response to a user query using the BM25 ranking algorithm. It provides a flexible and adaptable solution for the creation of question-answering systems and other applications that require the efficient retrieval of textual data.

6.4 Pipelines

We have developed 7 pipelines using the retrievers and readers described above.

6.4.1 Pipeline 01

In this pipeline we have used TF-IDF with cosine similarity as retriever and mBERT fine-tuned for question answering task as reader.

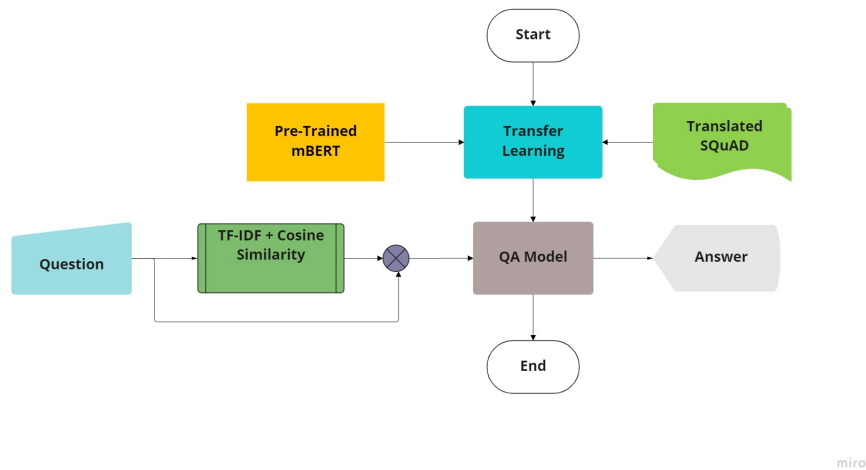


Figure 6.2: Pipeline 01

6.4.2 Pipeline 02

In this pipeline we have used TF-IDF with cosine similarity as retriever and DistilBERT fine-tuned for question answering task as reader.

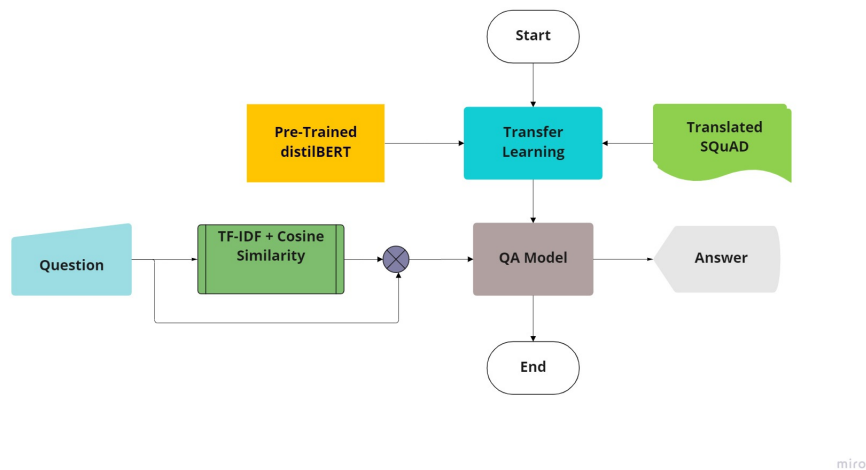
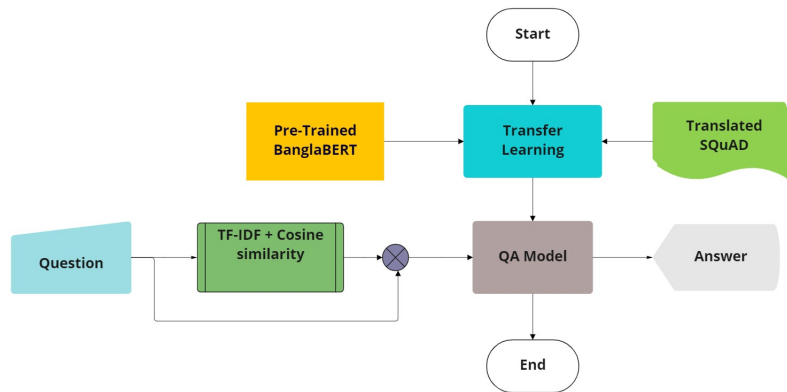


Figure 6.3: Pipeline 02

6.4.3 Pipeline 03

In this pipeline we have used TF-IDF with cosine similarity as retriever and BanglaBERT fine-tuned for question answering task as reader.

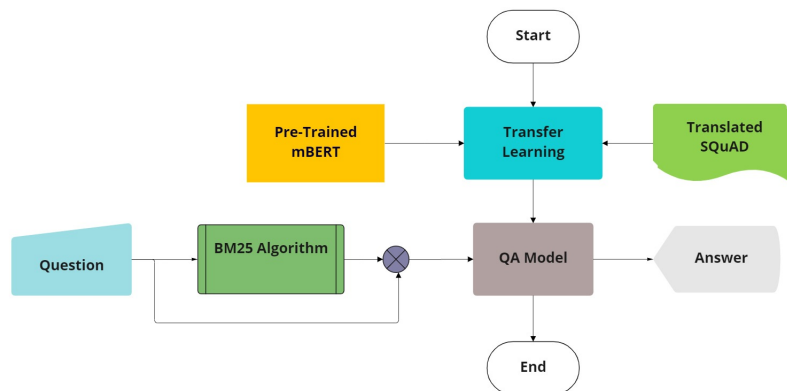


miro

Figure 6.4: Pipeline 03

6.4.4 Pipeline 04

In this pipeline we have used BM25 algorithm as retriever and mBERT fine-tuned for question answering task as reader.

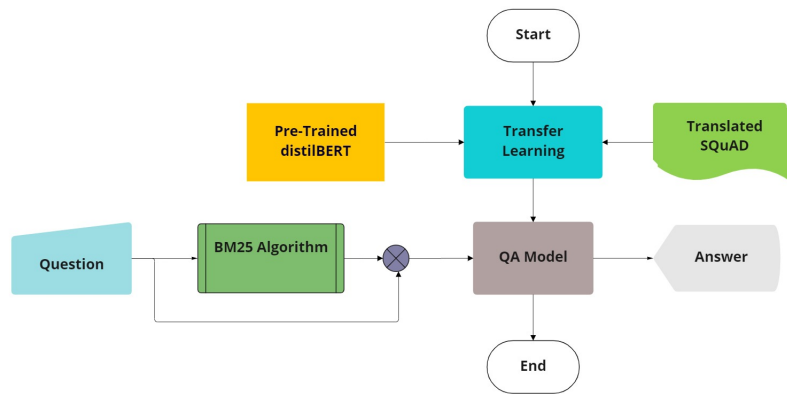


miro

Figure 6.5: Pipeline 04

6.4.5 Pipeline 05

In this pipeline we have used BM25 algorithm as retriever and DistilBERT fine-tuned for question answering task as reader.

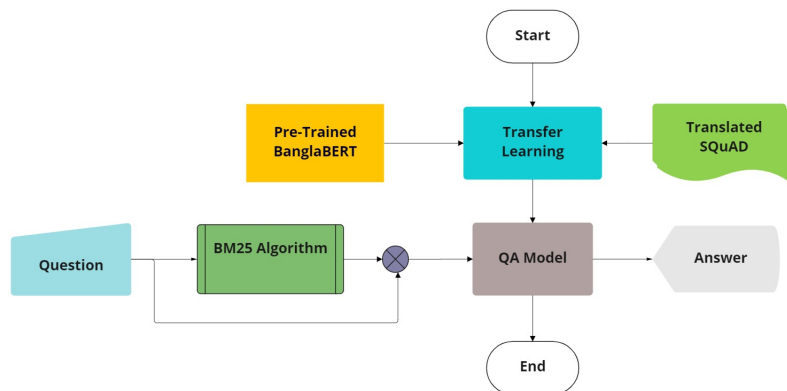


miro

Figure 6.6: Pipeline 05

6.4.6 Pipeline 06

In this pipeline we have used BM25 algorithm as retriever and BanglaBERT fine-tuned for question answering task as reader.

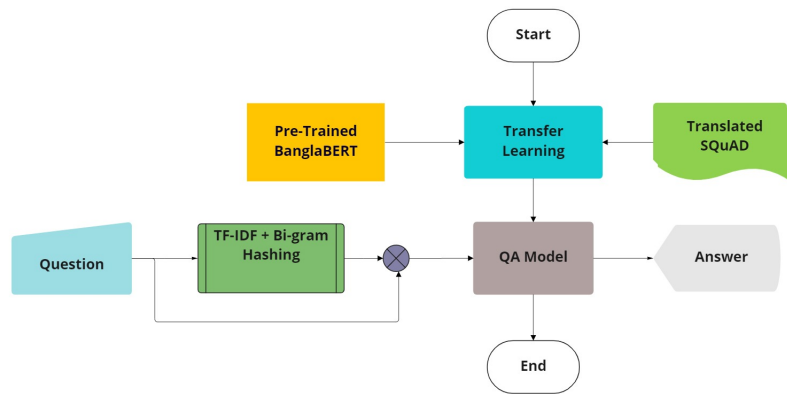


miro

Figure 6.7: Pipeline 06

6.4.7 Pipeline 07

In this pipeline we have used TF-IDF with Bi-gram hasing as retriever and BanglaBERT fine-tunned for question answering task as reader.



miro

Figure 6.8: Pipeline 07

Chapter 7

Results

Two metrics are used to evaluate the model's success on QA dataset : Exact Match (EM) and F1 accuracy. Each question and response is assigned a point value, which is then summed together. If the projected answer of the model has exactly the same characters as the actual answer, then $EM = 1$; otherwise, $EM = 0$. On this scale, being off by even one character results in a score of 0. The fraction $F1$ compares the word count of the expected and actual answers.

7.1 Fine Tuned BERT Models Result

We have fine-tuned three language models. Their details and train/loss graph are shown below.

7.1.1 Multilingual BERT

Table 7.1: Training results of Multilingual BERT

Training Loss	Epoch	Validation Loss
1.0407	1.0	1.1032

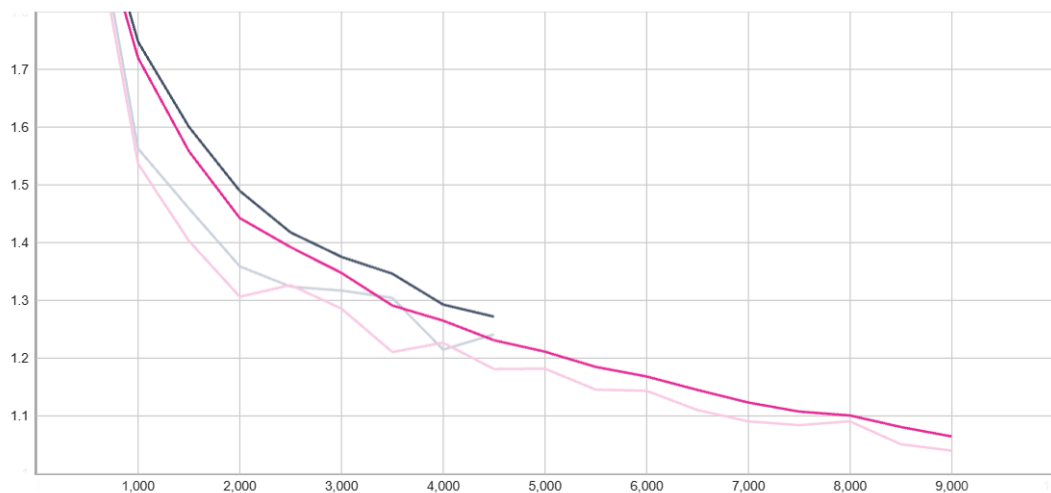


Figure 7.1: Train/loss of Fine-tuning Multilingual BERT.

7.1.2 DistilBERT

Table 7.2: Training results of DistilBERT

Training Loss	Epoch	Validation Loss
1.2709	1.0	1.4896

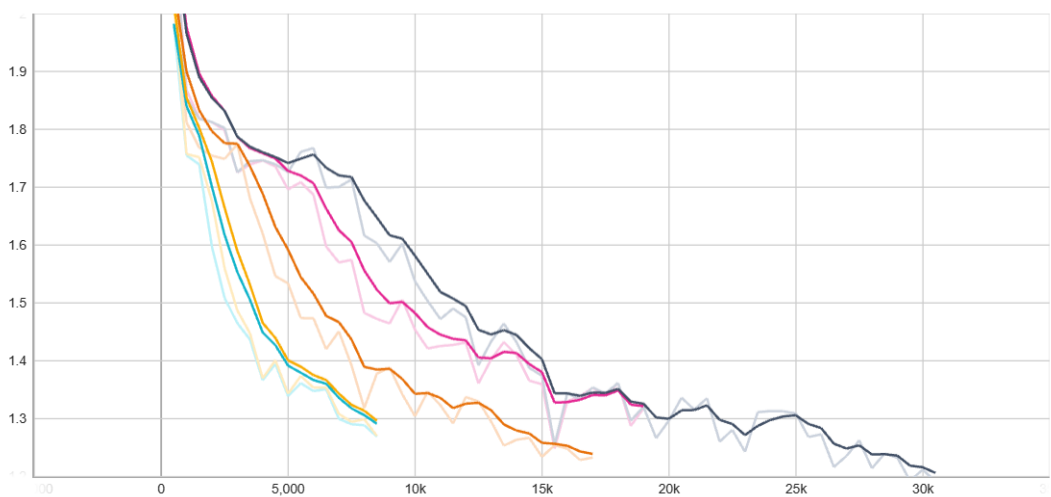


Figure 7.2: Train/loss of Fine-tuning DistilBERT.

7.1.3 BanglaBERT

Table 7.3: Training results of BanglaBERT

Training Loss	Epoch	Validation Loss
1.3649	1.0	1.4421

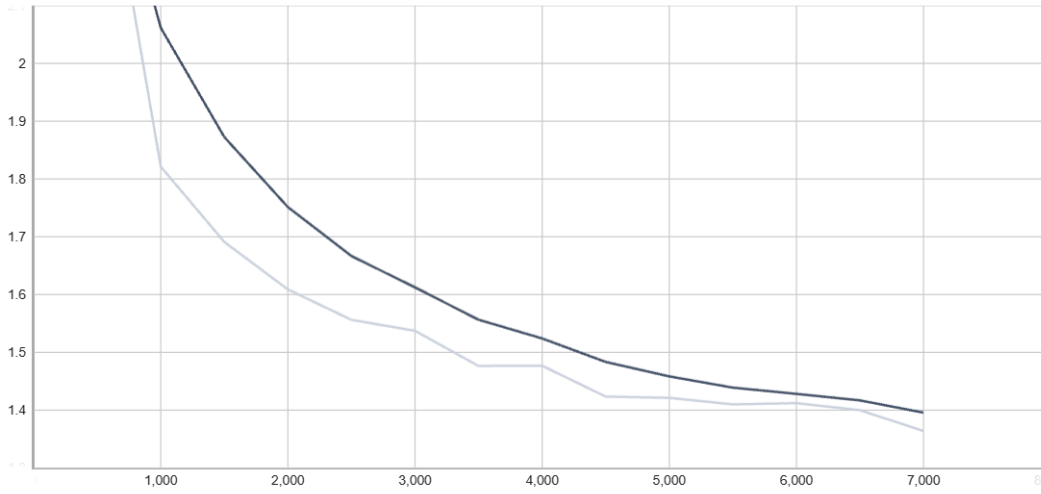


Figure 7.3: Train/loss of Fine-tuning BanglaBERT.

7.2 Retrievers Result

7.2.1 TF-IDF with Cosine Similarity

For single relevant documents, TF-IDF with Cosine Similarity had a Recall of 0.494 and for multiple relevant documents, it had a Recall of 0.492. The system was able to retrieve relevant documents with a reasonably high ranking, as indicated by the Mean Reciprocal Rank of 0.336. The Precision value of 0.117 indicated that the algorithm returned a large number of irrelevant documents. The system performed well in retrieving relevant documents across numerous queries, as indicated by the Mean Average Precision of 0.33.

Table 7.4: TF-IDF with Cosine similarity retriever performance

Recall (single relevant document)	Recall (multiple relevant documents)	Mean Reciprocal Rank	Precision	Mean Average Precision
0.494	0.492	0.335	0.117	0.329

7.2.2 BM25

BM25 retriever component was able to accomplish a Recall of 0.838 for a single relevant document and 0.837 for multiple relevant documents, respectively. It can be deduced from the fact that the Mean Reciprocal Rank was 0.695 that the retriever was successful in retrieving pertinent documents that had a high ranking. The Precision, on the other hand, was only 0.213, which indicates that the system returned a significant number of documents that were not pertinent. Given that the Mean Average Precision was 0.682, it can be deduced that the retriever performed admirably in terms of its ability to retrieve pertinent documents in response to multiple queries.

Table 7.5: BM25 retriever performance

Recall (single relevant document)	Recall (multiple relevant documents)	Mean Reciprocal Rank	Precision	Mean Average Precision
0.838	0.837	0.695	0.213	0.682

7.2.3 TF-IDF with Bi-gram Hashing

TF-IDF with Bi-gram Hashing retriever component was able to accomplish a Recall of 0.613 for a single relevant document and 0.619 for multiple relevant documents, respectively.

Table 7.6: TF-IDF with Bi-gram Hashing retriever performance

Recall (single relevant document)	Recall (multiple relevant documents)
0.413	0.419

7.3 Pipelines Result

Table 7.7: All pipelines result

Retriever	Reader	EM	F1
TF-IDF with cosine similarity	mBERT fine-tuned	0.271	0.36
TF-IDF with cosine similarity	distilBERT fine-tuned	0.09	0.18
TF-IDF with cosine similarity	BanglaBERT fine-tuned	0.28	0.37
TF-IDF with bi-gram hashing	BanglaBERT fine-tuned	0.418	0.517
BM25	mBERT fine-tuned	0.47	0.618
BM25	distilBERT fine-tuned	0.167	0.333
BM25	BanglaBERT fine-tuned	0.491	0.623

Among the seven pipelines that were evaluated in the study, the one that utilized BM25 as the retriever and BanglaBERT fine-tuned with translated SQuAD dataset as the reader yielded the best results. Specifically, this pipeline achieved an exact match (EM) score of 49.1% and an F1 score of 62.3%, which are considered to be relatively high scores for the given task. Another approach that utilized BM25 as the retriever and mBERT fine-tuned with translated SQuAD dataset as the reader also showed promising results. However, it failed to surpass the performance of the aforementioned approach. The reason for this observation could be attributed to the fact that monolingual models, such as BanglaBERT, are more effective at understanding the nuances of the target language than multilingual models like mBERT. On the other hand, the performance of the remaining five pipelines was not as satisfactory. An example :

Asked Question : শাহজালাল বিজ্ঞান ও প্রযুক্তি বিশ্ববিদ্যালয়ের কেন্দ্রীয় লাইব্রেরি চালু হয় কত সালে ?

Predicted Answer : ১৯৯০ সালে

True Answer : ১৯৯০ সালে

Chapter 8

Conclusion

We explored several question-answering models in this research work, from the very beginning to the most recent models. We discover that the pretrained BanglaBERT model, fine-tuned with the translated version of the SQuAD 2.0 dataset, can produce good results. In addition, we proposed using a retriever to select the best matched context based on the user's question because BERT based QA requires two inputs, a Context and a Question. In our retriever-reader approach user only needs to ask a query to get the expected result no need to give a context as input. We achieved a score of 49.1% for Exact Match (EM) and 62.3% for F1 score for using BM25 algorithm as retriever and BanglaBERT fine-tuned for QA task as reader. To date, no such approach has been developed for bengali question-answering task. Also we collected 150 contexts related to SUST from the SUST website, Wikipedia, news portals, websites and created a dataset in the shape of the SQuAD dataset. Overall, our findings indicate that the retriever-reader strategy is a promising technique for developing close domain QA systems for the Bengali language. However, the performance of our system could be further enhanced by optimizing the hyperparameters and experimenting with various architectures for the retriever and reader components.

In the future, we hope to increase the size of our dataset. Also we plan to test our proposed system on other domains. By doing so, we will be able to assess the flexibility and versatility of our model, and identify any areas that require further refinement. We would like to develop a better retriever algorithm to extract the most relevant context according to the user's question and obtain the answer more efficiently. So that our retriever-reader pipeline QA model can serve the purpose of education, business, and voice-activated systems in any close domain QA task in Bengali language.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [2] A. Bhattacharjee, T. Hasan, W. U. Ahmad, K. Samin, M. S. Islam, A. Iqbal, M. S. Rahman, and R. Shahriyar, “Banglabert: Language model pretraining and benchmarks for low-resource language understanding evaluation in bangla,” *arXiv preprint arXiv:2101.00204*, 2021.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [4] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter,” *arXiv preprint arXiv:1910.01108*, 2019.
- [5] P. Rajpurkar, R. Jia, and P. Liang, “Know what you don’t know: Unanswerable questions for SQuAD,” in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Melbourne, Australia: Association for Computational Linguistics, Jul. 2018, pp. 784–789. [Online]. Available: <https://aclanthology.org/P18-2124>
- [6] Wikipedia, “List of languages by total number of speakers — Wikipedia, the free encyclopedia,” <http://en.wikipedia.org/w/index.php?title=List%20of%20languages%20by%20total%20number%20of%20speakers&oldid=1109139012>, 2022, [Online; accessed 15-September-2022].
- [7] M. Islam, “Research on bangla language processing in bangladesh: progress and challenges,” in *8th international language & development conference*, 2009, pp. 23–25.

- [8] W. A. Woods, "Progress in natural language understanding: an application to lunar geology," in *Proceedings of the June 4-8, 1973, national computer conference and exposition*, 1973, pp. 441–450.
- [9] B. F. Green Jr, A. K. Wolf, C. Chomsky, and K. Laughery, "Baseball: an automatic question-answerer," in *Papers presented at the May 9-11, 1961, western joint IRE-AIEE-ACM computer conference*, 1961, pp. 219–224.
- [10] E. Riloff and M. Thelen, "A rule-based question answering system for reading comprehension tests," in *ANLP-NAACL 2000 Workshop: Reading Comprehension Tests as Evaluation for Computer-Based Language Understanding Systems*, 2000. [Online]. Available: <https://aclanthology.org/W00-0603>
- [11] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "Squad: 100,000+ questions for machine comprehension of text," *arXiv preprint arXiv:1606.05250*, 2016.
- [12] M.-Y. Day, C.-C. Tsai, W.-C. Chuang, J.-K. Lin, H.-Y. Chang, T.-J. Sun, Y.-J. Tsai, Y.-H. Chiang, C.-Z. Han, W.-M. Chen *et al.*, "Imtku question answering system for world history exams at ntcir-12 qa lab2." in *NTCIR*, 2016.
- [13] Y. Liu, B. Xu, Y. Yang, T. Chung, and P. Zhang, "Constructing a hybrid automatic q&a system integrating knowledge graph and information retrieval technologies," in *Foundations and trends in smart learning*. Springer, 2019, pp. 67–76.
- [14] D. Diefenbach, S. Amjad, A. Both, K. Singh, and P. Maret, "Trill: A reusable front-end for qa systems," in *European Semantic Web Conference*. Springer, 2017, pp. 48–53.
- [15] A. Abdi, N. Idris, and Z. Ahmad, "Qapd: an ontology-based question answering system in the physics domain," *Soft Computing*, vol. 22, no. 1, pp. 213–230, 2018.
- [16] L. Siciliani, P. Basile, G. Semeraro, and M. Mennitti, "An italian question answering system for structured data based on controlled natural languages." in *CLiC-it*, 2019.
- [17] M. Ketsmur, M. Rodrigues, and A. Teixeira, "Dbpedia based factual questions answering system." *IADIS International Journal on WWW/Internet*, vol. 15, no. 1, 2017.

- [18] A. A. Gunawan, P. R. Mulyono, and W. Budiharto, "Indonesian question answering system for solving arithmetic word problems on intelligent humanoid robot," *Procedia Computer Science*, vol. 135, pp. 719–726, 2018.
- [19] G. S. Dhanjal, S. Sharma, and P. K. Sarao, "Gravity based punjabi question answering system," *International Journal of Computer Applications*, vol. 147, no. 3, p. 21, 2016.
- [20] T. S. Walia, G. S. Josan, and A. Singh, "An efficient automated answer scoring system for punjabi language," *Egyptian Informatics Journal*, vol. 20, no. 2, pp. 89–96, 2019.
- [21] S. Banerjee and S. Bandyopadhyay, "Bengali question classification: Towards developing qa system," in *Proceedings of the 3rd Workshop on South and Southeast Asian Natural Language Processing*, 2012, pp. 25–40.
- [22] M. A. Islam, M. F. Kabir, K. Abdullah-Al-Mamun, and M. N. Huda, "Word/phrase based answer type classification for bengali question answering system," in *2016 5th International Conference on Informatics, Electronics and Vision (ICIEV)*. IEEE, 2016, pp. 445–448.
- [23] S. M. H. Nirob, M. K. Nayeem, and M. S. Islam, "Question classification using support vector machine with hybrid feature extraction method," in *2017 20th International Conference of Computer and Information Technology (ICCIT)*. IEEE, 2017, pp. 1–6.
- [24] S. Khan, K. T. Kubra, and M. M. H. Nahid, "Improving answer extraction for bangali q/a system using anaphora-cataphora resolution," in *2018 International Conference on Innovation in Engineering and Technology (ICIET)*. IEEE, 2018, pp. 1–6.
- [25] S. Banerjee, S. K. Naskar, P. Rosso, and S. Bndyopadhyay, "Classifier combination approach for question classification for bengali question answering system," *Sādhanā*, vol. 44, no. 12, pp. 1–14, 2019.
- [26] S. Sarker, S. T. A. Monisha, and M. M. H. Nahid, "Bengali question answering system for factoid questions: A statistical approach," in *2019 International Conference on Bangla Speech and Language Processing (ICBSLP)*. IEEE, 2019, pp. 1–5.
- [27] E. Perez, P. Lewis, W.-t. Yih, K. Cho, and D. Kiela, "Unsupervised question decomposition for question answering," *arXiv preprint arXiv:2002.09758*, 2020.

- [28] K. Kafle, B. Price, S. Cohen, and C. Kanan, “Dvqa: Understanding data visualizations via question answering,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 5648–5656.
- [29] M. Kowsher, M. M. Rahman, S. S. Ahmed, and N. J. Prottasha, “Bangla intelligence question answering system based on mathematics and statistics,” in *2019 22nd International Conference on Computer and Information Technology (ICCIT)*. IEEE, 2019, pp. 1–6.
- [30] S. T. Islam and M. N. Huda, “Design and development of question answering system in bangla language from multiple documents,” in *2019 1st International Conference on Advances in Science, Engineering and Robotics Technology (ICASERT)*. IEEE, 2019, pp. 1–4.
- [31] L. Hirschman, M. Light, E. Breck, and J. D. Burger, “Deep read: A reading comprehension system,” in *Proceedings of the 37th Annual Meeting of the Association for Computational Linguistics*. College Park, Maryland, USA: Association for Computational Linguistics, Jun. 1999, pp. 325–332. [Online]. Available: <https://aclanthology.org/P99-1042>
- [32] S. Reddy, D. Chen, and C. D. Manning, “Coqa: A conversational question answering challenge,” *CoRR*, vol. abs/1808.07042, 2018. [Online]. Available: <http://arxiv.org/abs/1808.07042>
- [33] A. Trischler, T. Wang, X. Yuan, J. Harris, A. Sordoni, P. Bachman, and K. Suleman, “NewsQA: A machine comprehension dataset,” in *Proceedings of the 2nd Workshop on Representation Learning for NLP*. Vancouver, Canada: Association for Computational Linguistics, Aug. 2017, pp. 191–200. [Online]. Available: <https://aclanthology.org/W17-2623>
- [34] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer, “TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension,” in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vancouver, Canada: Association for Computational Linguistics, Jul. 2017, pp. 1601–1611. [Online]. Available: <https://aclanthology.org/P17-1147>
- [35] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. Bowman, “GLUE: A multi-task benchmark and analysis platform for natural language understanding,” in *Proceedings of*

the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP. Brussels, Belgium: Association for Computational Linguistics, Nov. 2018, pp. 353–355. [Online]. Available: <https://aclanthology.org/W18-5446>

- [36] P. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich, “Superglue: Learning feature matching with graph neural networks,” *CoRR*, vol. abs/1911.11763, 2019. [Online]. Available: <http://arxiv.org/abs/1911.11763>
- [37] M. M. Uddin, N. S. Patwary, M. M. Hasan, T. Rahman, and M. Tanveer, “End-to-end neural network for paraphrased question answering architecture with single supporting line in bangla language,” *International Journal of Future Computer and Communication*, vol. 9, no. 3, 2020.
- [38] “Shahjalal University of Science Technology,” <https://www.sust.edu/about>, [Online; accessed 2022-09-17].